

A Method for Improving Retrieval Effectiveness in RAG

Xuwen Zhou^{*†}, Zhuowei Li^{*}, Rong Jiang^{*‡}, Chaojun Xu^{*} and Chengcheng Shao[§]

^{*}College of Computer Science and Technology, National University of Defense Technology, Changsha, China

¹603723357@qq.com, ²2549136534@qq.com, ³jiangrong@nudt.edu.cn, ⁴735158082@qq.com

[†]Unit 31306 of PLA, Chengdu, China

[§]College of Meteorology and Oceanology, National University of Defense Technology, Changsha, China

¹shaoc@nudt.edu.cn

Abstract—To address the performance fluctuation issues in Retrieval-Augmented Generation (RAG) frameworks, this paper proposes a collaborative optimization method integrating hybrid retrieval, grid search parameter optimization, and recursive hierarchical retrieval. The method dynamically adjusts weights to balance contributions from keyword-based sparse retrieval (BM25) and semantic dense retrieval (DPR). Combined with grid search for global optimization of hybrid weights, it significantly improves text recall across diverse query scenarios. Furthermore, a recursive retrieval strategy is introduced to enhance coverage of long-tail knowledge and complex structured documents through multi-granularity parsing and progressive retrieval augmentation. Experimental results on the Amnesty_qa dataset demonstrate that hybrid retrieval achieves over 20% recall improvement compared to single dense retrieval, while recursive retrieval boosts precision by 4.61%, validating the effectiveness of the proposed approach.

Index Terms—Retrieval-Augmented Generation, Hybrid Retrieval, Recursive Retrieval, Semantic Recall Rate

I. INTRODUCTION

Breakthroughs in the field of natural language processing are largely due to the leaps in semantic understanding, logical reasoning, and text generation capabilities of large language models. However, when faced with tasks that require real-time or expertise, such as open-domain Q&A, fact-proofing, and knowledge-intensive conversations, the limitations of traditional Large Language Models (LLMs) relying on parametric memory are becoming increasingly prominent:

On the one hand, the “time freeze” nature of model training data leads to insufficient coverage of the latest events or domain dynamics;

On the other hand, parametric memory makes the generation easily affected by noise or contradictory information in the training data, causing the problem of “hallucination”.

In addition, for application scenarios that require interpretable evidence support (such as medical diagnosis and legal judgments), LLMs are difficult to provide a clear path of “knowledge traceability”. In this context, the RAG framework

has become the core path to solve the above challenges by virtue of the collaborative paradigm of “external knowledge base and generative model”. The innovation of the RAG framework lies in its clever integration of two memory modes: one is the parametric memory of the pre-trained model, which contains the general knowledge and language patterns in the model weights; the other is the non-parametric memory of external knowledge sources, which exists in the form of massive text documents. RAG not only alleviates the lag problem of knowledge update by retrieving external knowledge bases in real time, but also enhances the credibility of generated content through traceable search results.

The typical processes of RAG include [1]:

- **Retrieval stage:** recall relevant document fragments from external knowledge bases based on user queries, i.e., for user queries (Query), first use an efficient retriever to find the most relevant document fragments in real time from a huge external knowledge base (such as Wikipedia, professional literature library, corporate documents, etc.);
- **Generation stage:** Stitch the query and search results into the LLMs to generate the output of fusion knowledge.

This mechanism gives the RAG framework excellent knowledge acquisition and fusion capabilities. It is no longer limited to the knowledge “frozen” during model training, but can dynamically and specifically extract the required information from the external knowledge ocean [2], thereby significantly improving the accuracy, timeliness, factual consistency and logical rigor of the generated content, making it more in line with human knowledge cognition and reasoning process [3].

Despite the significant advantages of RAG, existing methods still face two core challenges:

- Firstly, insufficient semantic alignment between user queries and retrieval knowledge. The current mainstream search strategies are mainly divided into two categories: sparse search (such as BM25) relies on keyword matching, which is computationally efficient but has limited ability to capture semantic associations; Dense retrieval (such as Sentence-BERT) realizes vector similarity matching through semantic coding, which improves semantic correlation but faces the disadvantages of high

[‡]Corresponding author. This work was supported by the National Key Research and Development Program of China (Grant No. 2023YFC3306104) and the National Natural Science Foundation of China No. 62202487.

computational complexity and insufficient coverage of long-tail knowledge.

- Secondly, the balance between retrieval complexity and effect optimization. Existing studies have insufficient exploration of multi-granularity knowledge fusion and dynamic parameter tuning. It is particularly prominent in query scenarios with dense technical terms and complex logical chains, and the requirements for retrieval strategies vary significantly in different query scenarios, making it difficult to achieve global optimization with mixed retrieval or static parameter settings with fixed weights.

The main work of this paper is summarized as follows:

(1) **Dynamic Weight Adjustment for Hybrid Retrieval Strategies:** A dynamic weight allocation mechanism based on a dual-score system is designed to balance the contributions of sparse and dense retrieval, while considering both semantic relevance and keyword matching efficiency;

(2) **Grid search-driven global parameter optimization:** global tuning of mixed weights based on reinforcement learning strategy to adapt to the optimal configuration of different query scenarios;

(3) **Multi-granularity enhancement of recursive hierarchical retrieval:** Introducing a multi-stage retrieval process to improve the coverage of long-tail knowledge and complex structured documents through the gradual strategy of “coarse-grained parsing-fine-grained positioning”.

The remainder of this paper is organized as follows. In Section II, we will introduce the related work. In Section III, we present our proposed architecture in detail, followed by experiments and results analysis in section IV. Finally, we draw our conclusion and in Section V.

II. RELATED WORK

The Facebook Research (FAIR) team first proposed the concept of RAG in 2020 and detailed the basic architecture and application of RAG [4]. Retrieval-augmented generation is a model that combines retrieval and generation techniques. It enhances the generation ability of the language model by retrieving relevant information from an external knowledge base, thereby improving the quality and relevance of the generated content.

REPLUG [5] treats the language model as a “black box,” using a retriever to call external text, then constructing prompt sequences to seamlessly integrate relevant external text into the query, and finally inputting all the integrated text into the “black box” to generate a response. ECBRF [6] effectively improves the performance and accuracy of question answering systems by combining query and retrieval information into prompts. BLLM augmentation [7] is a method of integrating retrieved information directly into model inputs without the need to train a RAG generator with additional samples.

The basic process of the RAG model is to first retrieve the input query, obtain the relevant documents or information fragments, and then input the retrieved information into the generator together along with the query to generate the final

output [8]. Conventional RAG systems tend to perform well when the document is clearly relevant to the context of the question [9]. However, if the document contains only partial information or is less obvious to the context, the performance of RAG will decrease significantly.

Shi et al. [10] and Gao et al. [11] pointed out that the current RAG lacks highly specialized training in retrieving information, resulting in biases in content selection and response. In-Context RALM [12] uses the BM25 algorithm for document retrieval and reordering to select documents relevant to input integration. Karpukhin et al. [13] proposed a dense passage retriever (DPR) model, which generates dense vector representations through query encoders and paragraph encoders based on the BERT model, and calculates correlation scores using query vectors and text vectors.

In order to compensate for the shortcomings of sparse search and dense search, Wang et al. [14] used a hybrid retriever composed of BM25 sparse search method and dense search model for document retrieval, and found that the use of hybrid search was more capable of recalling text than using sparse search or dense search alone. Pradeep [15] and Huang [16] tried to improve document ranking by combining searchers with reorderers on the basis of hybrid search. Asai et al. [17] proposed a self-reflective RAG strategy that improves the effectiveness of generating text by introducing dynamic retrieval and feedback mechanisms, but also significantly increases computational resources. Xu [18] and Dhuliawala [19] attempts to optimize the retrieval strategy of large language models through prompt engineering, and explore under what scenarios and when parameters are optimal. Although these prompting techniques can guide the model’s retrieval behavior to a certain extent, they fail to fundamentally change the model’s ability to utilize the retrieved text.

III. PROPOSED ARCHITECTURE

The model architecture proposed in this paper is shown in the framework in Figure 1. It is mainly composed of a hybrid retrieval recall module and a grid search optimization module. The hybrid retrieval recall module combines the sparse vector retrieval and dense vector retrieval methods. Sparse vector retrieval uses the BM25 algorithm modified by TF-IDF, which is a keyword retrieval algorithm based on word frequency statistics. Dense vector retrieval uses the BERT pre-trained model as the vector model. Grid search optimization continuously optimizes this model in a loop to determine the optimal weighted average parameters for hybrid retrieval.

The specific process of this method is to first use the vector model to convert the text knowledge base into a dense vector library, and then vectorize the query question (query), and then calculate the cosine similarity of the vector to get the score of the vector retrieval.

This article also uses the BM25 algorithm to perform sparse retrieval on queries to obtain the keyword retrieval score. Finally, the entire model is optimized through grid search through the hyperparameter tuning method, so that it can continuously perform retrieval and optimization, and select

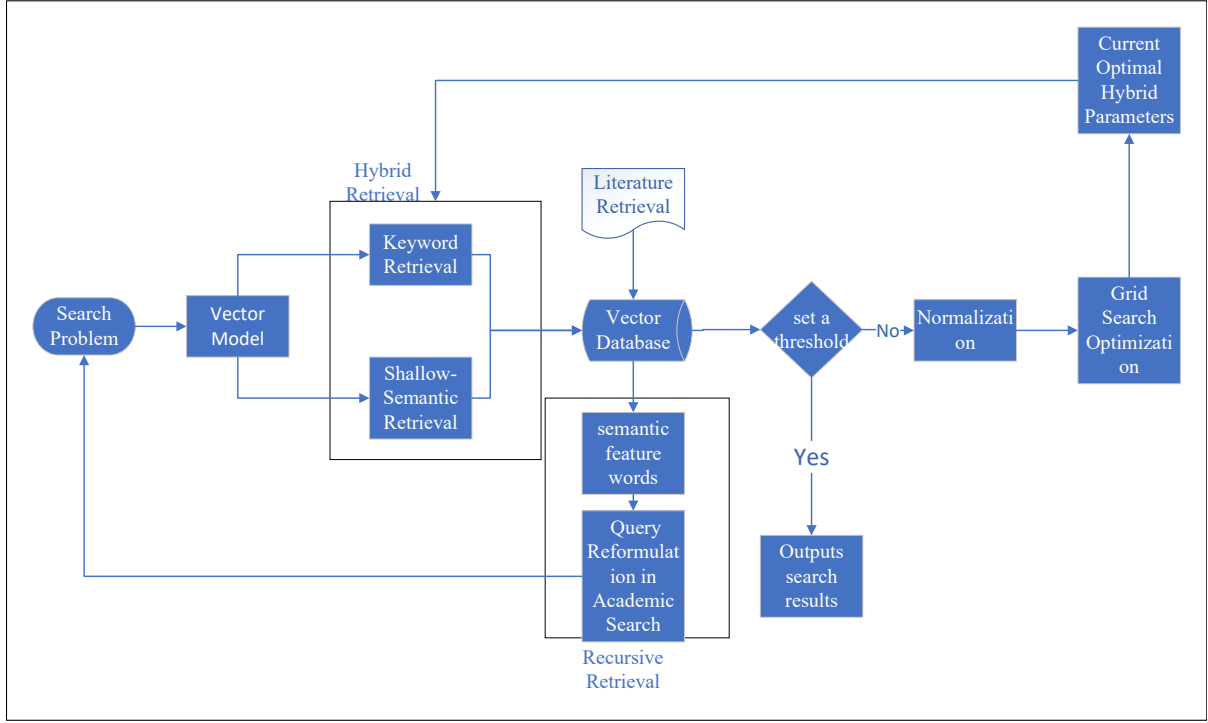


Fig. 1. The overall architecture of the proposed collaborative optimization method.

the best hybrid retrieval weighted average parameter . This parameter is used to normalize the scores obtained by the above two methods to obtain the final score of the hybrid retrieval, and the top k text blocks are returned as the final result of the retrieval based on the optimal score. The specific design of the hybrid retrieval recall module and the grid search optimization module will be introduced in detail in the next two sections.

A. Hybrid Retrieval Module

The hybrid retrieval recall module combines BM25 and vector retrieval based on a large language model, and adopts a dual-path scoring system to significantly improve the recall rate of retrieval. The BM25 algorithm is a retrieval algorithm based on a probability model, which calculates the relevance between query keywords and documents by counting word frequency and document length. The core idea of BM25 lies in its inverted index, which records the document frequency and word position information of each term, and introduces the length information of the document to calculate the relevance between the query and the document. For the query word set $Q = q_1, q_2, \dots, q_n$ and document D , the calculation formula for its relevance score is shown in formula (1):

$$BM25(D, Q) = \sum_{i=1}^n IDF(q_i) \times \frac{f(q_i, D) \times (K_1 + 1)}{f(q_i, D) + K_1 \times \left(1 - b + b \times \frac{|D|}{avgdl}\right)} \quad (1)$$

Where $BM25(D, Q)$ is the BM25 relevance score of query Q and document D , $IDF(q_i)$ is the inverse document frequency of term q_i , $f(q_i, D)$ is the frequency of term q_i in document D , $|D|$ is the document length, $avgdl$ is the average document length of the corpus, K_1 and b are adjustment parameters, The value ranges are $K_1 \in [1.2, 2.0]$, $b \in [0, 1]$.

The calculation formula of IDF is shown in formula (2):

$$IDFq_i = \log\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}\right) \quad (2)$$

Where N is the total number of documents, and $n(q_i)$ is the number of documents containing q_i .

Vector retrieval based on the BERT pre-trained model uses a large model to vectorize queries and documents respectively, and obtains the score of vector retrieval by calculating the cosine similarity between the query and the document. The calculation formula of vector cosine similarity is shown in formula (3):

$$cosine_{similarity}(vQ, vD) = vQ \times vD = \sum (vQ_i \times vD_i) \quad (3)$$

Where $vQ = Encoder(Q)$, $vD = Encoder(D)$.

This module finally uses the weight parameter of the vector retrieval score to perform a weighted average of the above two scoring results, sorts the retrieval results according to the final hybrid retrieval score, and returns the top K text blocks. The hybrid retrieval score calculation formula is shown in formula (4):

$$final_{score} = \alpha \times norm(cosine_{sim}) + (1 - \alpha) \times norm(BM(25)_{score}) \quad (4)$$

B. Grid Search Optimization Module

The grid search optimization module adopts a reinforcement learning strategy, systematically evaluates the retrieval effect under different weight parameters through RAGAS, and automatically finds the optimal weight parameter in the hybrid retrieval score. The optimal parameter found by this module can better balance the contribution of semantic-based vector retrieval and BM25 retrieval, ensuring that the hybrid retrieval module achieves the best recall effect.

The initial parameter space is preset by the system. The default setting of α is in the range of [0.0, 1.0], with a step size of 0.1. It can be adjusted according to actual needs. The mathematical expression of the parameter space is as shown in formula (5):

$$\alpha \in \{x | x = 0.0 + 0.1 \times k, k = 0, 1, \dots, 10\} \quad (5)$$

This module designs a two-stage progressive search strategy:

the first stage performs coarse-grained screening on the initial parameter space to quickly lock the optimal parameter interval;

the second stage switches to 0.01 steps within the locked interval for fine tuning according to empirical values, and combines the early stopping mechanism to stop when the optimization amplitude is less than 0.5% for three consecutive times to improve the efficiency of grid search.

This module, as an auxiliary optimization module of the hybrid retrieval module, also includes document vectorization and evaluation steps, and has certain requirements on computing resources.

C. Recursive Search Module

The framework realizes self-iterative optimization of the retrieval process through a dynamic feedback mechanism, and includes two core components: the basic retrieval module and the recursive retrieval module.

The basic retrieval module is the atomic operation unit of recursive retrieval, and adopts a hybrid retrieval architecture to implement a single retrieval process. Given a query Q, the module performs the following calculation process:

Step 1, calculate the score of the hybrid retrieval, refer to formula (4).

Step 2, query expansion preprocessing, build a dynamic vocabulary W based on the current retrieval result $R\{r_1, r_2, \dots, r_n\}$.

Step 3: Calculate keyword weights and select expansion words by word frequency-inverse document frequency.

Step 4: Select expansion words and select the m non-stop words with the highest weights to form the expansion word set E. $E = \{w_j \in W \setminus S | Score(w_j)\}$, Sort the first m bits in descending order. S is the inherent stop word list, $Score(w_j)$ is the expanded word weight score, and its calculation formula is shown in formula (6).

$$Score(w_j) = \frac{count(w_j, R)}{|R|} * \log\left(\frac{N + 1}{n(w_j) + 1}\right) \quad (6)$$

Where $count(w_j, R)$ is the total number of occurrences of term w_j in the search result set R, $|R|$ is the total number of documents in the current retrieval result, and N is the total number of documents in the knowledge base.

This module directly improves the precision of RAG retrieval by using the extended vocabulary E obtained by the basic retrieval module as auxiliary information for the next retrieval to reduce the noise information in the retrieval results. The improvement of retrieval precision can reduce the illusion phenomenon and irrelevant information in the generation link. This module sets the maximum number of recursions to two times. This setting is mainly based on the consideration that too many recursions will increase the time cost of retrieval and reduce the integrity of the retrieved relevant text.

D. System Design and Implementation

In view of the collaborative optimization framework of integrated hybrid retrieval, grid search parameter optimization and recursive hierarchical retrieval proposed in the previous article, this paper constructs a complete technical module from the dimensions of module function division, technical implementation path, interactive logic design and key parameter configuration to ensure that various technical strategies form an organic whole in research practice and achieve a dual improvement in retrieval effect and engineering feasibility. As the core input processing unit of the system, the hybrid retrieval module undertakes the key function of converting user queries into structured retrieval signals. This module adopts a dual-path architecture that combines keyword sparse retrieval with semantic dense retrieval, and achieves coverage of diverse query needs through precise matching of technology selection and deep optimization of process design.

The core processing flow of the hybrid retrieval module includes document preprocessing and index construction, two-way parallel scoring, and score normalization and mixing. In the process of document preprocessing and index construction, the system first divides the original text into blocks according to natural semantic units, and uses sliding window technology to cut long texts into segments of about 300 words in length. Use the pre-trained BERT-base-cased model to encode each Token sequence, and extract the vector of the first token [CLS] of the last hidden state, which is usually used as the semantic representation of the entire sentence/paragraph. As the dimensional dense semantic vector of the document block. Then build an efficient vector retrieval library based on FAISS to provide underlying data support for subsequent semantic retrieval. At the same time, build a traditional inverted index

for BM25. The two-way parallel scoring mainly processes the input query Q through two paths.

As the core of intelligent tuning of the system, the grid search optimization module undertakes the task of global optimization of the dynamic weight parameter. Its design goal is to find the optimal solution that maximizes the retrieval effect within the preset parameter space. This module adopts a three-stage strategy of “coarse-grained screening – fine-tuning optimization – early stopping mechanism” to effectively control the computational cost while ensuring the optimization accuracy. The parameter space is defined as $[0.0, 1.0]$. In the initial stage, a coarse-grained step size of 0.1 is used to divide the parameter space into 11 discrete points, covering the complete spectrum from pure sparse retrieval ($\alpha = 0$) to pure dense retrieval ($\alpha = 1$). This division method not only ensures the comprehensiveness of global exploration, but also avoids computational redundancy caused by excessive subdivision.

In order to solve the problem of missed detection in long-tail knowledge retrieval, the recursive retrieval module achieves in-depth coverage of complex document structures and low-frequency knowledge through a three-layer architecture of “multi-granularity parsing – progressive expansion – intelligent merging”. The core innovation of the module lies in the organic combination of the extended vocabulary generation mechanism and the recursive retrieval process, dynamically expanding the query semantics through statistical learning methods to form a step-by-step in-depth retrieval capability. The extended vocabulary generation process is divided into two key steps: the first is the word frequency statistics, which segment the Top-K documents returned by the initial search, use the NLTK stop word list to filter common words, and count the frequency of occurrence of the remaining terms in the result set; the second is the weight calculation, which uses the TF-IDF method, combined with local frequency (TF) and global rarity (IDF), to screen out extended words that appear frequently in the result set and have domain specificity, to ensure the semantic relevance of the extended vocabulary E .

The recursive retrieval process is divided into four stages: the first stage is the initial retrieval, which uses the original query to call the hybrid retrieval module to obtain the Top-K1 (such as $K_1 = 100$) documents as the first round of knowledge screening results; the second stage is query expansion, which merges the top m (default $m = 5$) words in the expanded vocabulary E with the original query, and generates a compound query through grammatical rules (such as “original query + AND + expanded word 1 + AND + expanded word 2”) to achieve explicit expansion of query semantics; the third stage is secondary retrieval, which uses the compound query to perform hybrid retrieval again, obtains Top-K2 (such as $K_2 = 100$) documents, and merges them with the initial retrieval results to avoid repeated retrieval; the fourth stage is re-ranking, which uses the same scoring formula as the hybrid retrieval for the merged result set, combined with BM25 and BERT scores for re-ranking, and outputs the final Top-K (such as $K = 50$) documents. In this article, the maximum number of iterations of the recursive process is set to two times. This

setting achieves the best balance between the improvement of recall rate and time cost, and avoids performance degradation caused by excessive recursion.

IV. EXPERIMENTS AND RESULTS ANALYSIS

This paper improves the retrieval method based on RAG technology and builds a hybrid retrieval system and a recursive retrieval system. The main hardware and software configurations used in the construction and training process of the designed model are shown in Table I.

TABLE I
SOFTWARE AND HARDWARE CONFIGURATION

Programming Language	PyTorch 2.6.0
Deep Learning Frameworks	final.hta
Device Memory	32G
CPU Processor	Intel(R) Core(TM) i9-14900HX 2.20 GHz
GPU Graphics Card	NVIDIA GeForce RTX 4060
Operating System	Windows 12

A. Dataset Introduction

The public datasets used in this experiment are mainly the Amnesty_qa human rights data and environmental responsibility datasets and the HotpotQa development set. The Amnesty_qa dataset is a Grounded QA dataset created based on a research report by the human rights organization Amnesty International. It contains fields such as questions, correct answers, and contextual information. The dataset construction process includes extracting key information from the report and generating related question and answer pairs. In this way, the dataset not only covers a wide range of human rights issues, but also ensures the accuracy and authority of the information. The dataset is characterized by its multilingual support, covering multiple languages such as English, Malayalam, and Hindi, enabling it to serve a wider user group. In addition, each question in the dataset is accompanied by detailed background information and reference answers, ensuring the richness and practicality of the data. The structure of the dataset is clear, which makes it easy for users to quickly locate the required information. Therefore, this dataset can better meet the needs of RAGAS evaluation, and its answer field can be treated as a necessary reference field for RAGAS without unnecessary changes.

TABLE II
DATASET DETAILS

Dataset Name	Data Description	Total Amount of Data
HotPotQA	Multi-hop dataset	7405
Amnestyqa	Private Contracts	80

The HotpotQA development set contains about 7,400 question-answer pairs. Among them, its supporting_facts field is manually annotated information, that is, a list of sentences

supporting reasoning. It is not directly given in the HotpotQA dataset, but the position information in the context is given in the form of document title and sentence number. Therefore, in order to facilitate the evaluation of the RAGAS method adopted in this study, the development set data is processed accordingly, mainly by extracting the specific text content of supporting_facts through indexing as the ground_truth field required for RAGAS evaluation, which is a key indicator required for calculating context_recall. The type field is removed during processing as irrelevant information for RAGAS evaluation, and the remaining fields remain unchanged.

B. Evaluation Metrics

This paper uses the RAGAS evaluation framework to quantitatively evaluate the context recall capability of the retrieval system. Based on the characteristics of the retrieval enhancement generation system, context_recall is selected as the core evaluation indicator, and its calculation logic can be expressed as:

$$\text{Context Recall} = \frac{|RelevantContexts \cap RetrivedContexts|}{|RetrivedContexts|} \quad (7)$$

Where *RelevantContexts* is the set of standard answer documents, *RetrivedContexts* is the set of system retrieval results, and $|R|$ represents the cardinality of set *R*.

When using RAGAS, developers can quickly integrate through Python API or command line tools. For example, calling `ragas.evaluate (dataset, metrics=[faithfulness, context_relevance])` can start the evaluation and export a detailed report in JSON format. The framework supports seamless integration with tool chains such as LangChain and LlamaIndex, and provides open source code and pre-trained models to help users quickly locate bottlenecks and iterate models, ultimately achieving a coordinated improvement in retrieval accuracy and generation reliability.

C. Experimental Results Analysis

According to the experimental design, a total of three groups of experimental data were obtained. The first group of experiments evaluated and tested the BM25 retrieval only, the dense retrieval based on the BERT pre-training model only, the hybrid retrieval based on the empirical value parameter ($\alpha=0.5$), and the hybrid retrieval based on grid search optimization. The experimental data are shown in Table III. On the Amnesty_qa dataset, the recall rate of the retrieval method that does not use retrieval optimization is lower than that of the hybrid retrieval, especially compared with the dense retrieval method, the recall rate of the hybrid retrieval has increased by more than 20%. Compared with the retrieval using the BM25 algorithm alone, the recall rates of the two groups of hybrid retrieval based on grid search optimization increased by 3.4% and 5.3% respectively, and there was a certain improvement in the completeness and accuracy of the retrieval results. This fully proves that the hybrid retrieval based on grid search optimization can optimize the quality of RAG retrieval.

TABLE III
COMPARISON OF RECALL BETWEEN MIXED AND SINGLE RETRIEVAL

Model Method	Recall Rate
BM25	74.20%
BERT-base-cased	52.74%
Bge-reranker	48.10%
BM25+Bgereranker($\alpha = 0.7$)	72.16%
BM25+BERT-base-cased($\alpha = 0.5$)	78.47%
BM25+Bge-reranker($\alpha = 0.7$)	77.70%

The results of the second set of experiments are shown in the table IV. Recursive retrieval has a significant improvement in precision compared to hybrid retrieval, reaching 3.9% and 4.61% respectively, while the recall rate has slightly decreased. Recursive retrieval can use the extended vocabulary generated by the basic retrieval module as auxiliary information to filter out irrelevant documents in the retrieval results, thereby improving the precision of the documents. The recall rate of recursive retrieval is still significantly improved by more than 18% compared to dense retrieval based on vector models. At the same time, it also significantly improves the precision of retrieval results and reduces the hallucination problem that may occur in the generation stage.

TABLE IV
COMPARISON OF RECURSIVE RETRIEVAL ACCURACY

Model Method	Accuracy
BM25+Bge-reranker($\alpha = 0.7$)	75.41%
BM25+BERT-base-cased($\alpha = 0.6$)	78.51%
Recursive Retrieval(Bge)	79.04%
Recursive Retrieval(BERT)	83.12%

The third set of experiments tested and evaluated the recall rate of text retrieval of hybrid retrieval on the HotpotQA dataset. Since the development set used in this study contains a large number of question-answer pairs (about 7,400), a block evaluation method is adopted in the evaluation process. The entire dataset is divided into multiple text blocks of 50 question-answer pairs, and the evaluation operation is performed independently on each block and the results are saved, which effectively avoids the timeout, memory overflow and API limitation problems that are common when processing large datasets at a time. In the specific implementation, the system first divides the original dataset into multiple consecutive blocks in sequence, and then starts the evaluation process for each block and adds an exponential backoff retry mechanism-the waiting time after the evaluation fails increases by a power of 2, and a maximum of 5 retries are made. The CSV format result file of the block is saved immediately after each successful evaluation, and a detailed error log is recorded in case of failure. Some test results are shown in Table V.

During the evaluation, timeouts, memory overflows, and API limitations still occurred on some question-answer pairs. After removing the number of empty values, the final scores

TABLE V
EXPERIMENT RESULT OF HOTPOTQA DATASET RECALL

Model	HotpotQa context_recall	Effective NO. of Data Entries
BM25+BERT-base-cased($\alpha = 0.5$)	61.09%	7300
BM25+BERT-base-cased($\alpha = 0.7$)	54.53%	7246

were counted. The text recall score when the parameter $\alpha = 0.5$ is about 6.56% higher than when $\alpha = 0.7$. There is still room for optimization of this parameter, but the optimization of hybrid retrieval parameters on large data sets is subject to certain hardware resource limitations, and grid search optimization takes too long. The retrieval performance of the hybrid retriever on HotpotQa is slightly inferior to its results on Amnesty_qa. This is because HotpotQa has a large number of multi-hop question-answer pairs, requiring the retriever to search for supporting facts across multiple questions and answers. The semantic retriever in the hybrid retrieval is still a shallow semantic retrieval, and it is difficult to completely solve this problem. However, compared with a single retrieval method, hybrid retrieval still performs well on this type of problem.

V. CONCLUSION

This paper optimizes the retrieval effect of the RAG system by constructing a hybrid retrieval system and a recursive retrieval system. Among them, the hybrid retrieval algorithm combines sparse retrieval based on BM25 with dense retrieval, taking into account the semantic accuracy of search results and the semantic generalization of key phrases. The results show that the recall rate (Recall5) of hybrid retrieval on the Amnesty_qa dataset is more than 20% higher than that of the single dense retrieval mode. On the one hand, sparse retrieval can meet some professional phrase hit requirements, and on the other hand, dense vector retrieval overcomes semantic inaccuracy such as semantic ambiguity and semantic context dependency. The static parameterized hybrid retrieval system cannot handle the diverse problems in different fields well. Therefore, this paper also implements dynamic hybrid retrieval weight parameters through grid search optimization, which enables the hybrid retrieval to better balance the weights between the two basic retrieval methods and ensures that the effect of the hybrid retrieval is optimized.

REFERENCES

- [1] M. Riedler and S. Langer, "Beyond text: Optimizing rag with multimodal inputs for industrial applications," *arXiv preprint arXiv:2410.21943*, 2024.
- [2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "Retrieval augmented language model pre-training," in *International conference on machine learning*. PMLR, 2020, pp. 3929–3938.
- [3] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Association for Computational Linguistics, 2021.

- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.
- [5] W. Shi, S. Min, M. Yasunaga, M. Seo, R. James, M. Lewis, L. Zettlemoyer, and W. T. Yih, "Replug: Retrieval-augmented black-box language models," in *2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2024*. Association for Computational Linguistics (ACL), 2024, pp. 8364–8377.
- [6] Z. Yang, X. Du, E. Cambria, and C. Cardie, "End-to-end case-based reasoning for commonsense knowledge base completion," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 2023, pp. 3509–3522.
- [7] M. Patidar, A. Singh, R. Sawhney, I. Bhattacharya *et al.*, "Combining transfer learning with in-context learning using blackbox llms for zero-shot knowledge base question answering," *arXiv.org*, 2023.
- [8] R. Feng, X. Hong, M. Jobanputra, M. Warning, and V. Demberg, "Retrieval-augmented modular prompt tuning for low-resource data-to-text generation," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 2024, pp. 14 053–14 062.
- [9] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, and B. Cui, "Retrieval-augmented generation for ai-generated content: A survey," *CoRR*, 2024.
- [10] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 210–31 227.
- [11] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," *arXiv preprint arXiv:2305.14627*, 2023.
- [12] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlga, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-context retrieval-augmented language models," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023.
- [13] V. Karpukhin, B. Oguz, S. Min, P. S. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *EMNLP (1)*, 2020, pp. 6769–6781.
- [14] S. Wang, S. Zhuang, and G. Zuccon, "Bert-based dense retrievers require interpolation with bm25 for effective passage retrieval," in *Proceedings of the 2021 ACM SIGIR international conference on theory of information retrieval*, 2021, pp. 317–324.
- [15] R. Pradeep, S. Sharifmoghaddam, and J. Lin, "Rankvicuna: Zero-shot listwise document reranking with open-source large language models," *arXiv preprint arXiv:2309.15088*, 2023.
- [16] C.-W. Huang and Y.-N. Chen, "Instupr: Instruction-based unsupervised passage reranking with large language models," *arXiv preprint arXiv:2403.16435*, 2024.
- [17] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," 2024.
- [18] S. Xu, L. Pang, H. Shen, X. Cheng, and T.-s. Chua, "Search-in-the-chain: Towards the accurate, credible and traceable content generation for complex knowledge-intensive tasks," *arXiv preprint arXiv:2304.14732*, 2023.
- [19] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," *arXiv preprint arXiv:2309.11495*, 2023.